String Distance, Similarity and Evaluation Metrics

Natural Language Processing

Daniel Ortiz Martínez

Table of Contents

Introduction

Measuring Similarity in Text: A Fundamental Challenge

How do we quantify how similar or different two pieces of text are?

Measuring Similarity in Text: A Fundamental Challenge

How do we quantify how similar or different two pieces of text are?

This seemingly simple question has different answers depending on:

- The **level** at which we compare (characters, words, semantics)
- The **task** we are trying to solve (spelling correction, duplicate detection, translation quality)
- The **computational resources** available

Why Multiple Approaches?

Example Pair:

- "The cat sat on the mat"
- "A kitten rested on the rug"

Why Multiple Approaches?

Example Pair:

- "The cat sat on the mat"
- "A kitten rested on the rug"

Different perspectives:

- Character-level: Very different
- Word-level: Almost no overlap
- Semantic: Very similar

Why Multiple Approaches?

Example Pair:

- "The cat sat on the mat"
- "A kitten rested on the rug"

Different perspectives:

- Character-level: Very different
- Word-level: Almost no overlap
- Semantic: Very similar

Key insight: Different metrics capture different notions of similarity. Choosing the right one depends on your application!

Summary of New Content

We will explore three levels of text comparison:

1. **String Distance Metrics**
   - Character-level operations (Levenshtein, Hamming, etc.)
   - Applications: spell checking, fuzzy matching, OCR correction
2. **Semantic Similarity**
   - Modern embedding-based approaches (SBERT)
   - Moving beyond lexical overlap to meaning
3. **NLP System Evaluation**
   - Task-specific metrics (BLEU, ROUGE, etc.)
   - Measuring the quality of generated text

String Distance

String Distance: Overview

String distance metrics measure the **dissimilarity** between two strings by counting the minimum number of operations needed to transform one string into another

Key characteristics:

- Operate at the **character level**
- Focus on **lexical similarity**, not semantic meaning
- Computationally efficient
- Language-agnostic (work with any alphabet)

Jaccard Distance

- Jaccard similarity: $s_{\text{jaccard}}(x, y) = \frac{|x \cap y|}{|x \cup y|}$

```
def jaccard_similarity(s1, s2):
    return len(s1.intersection(s2)) / len(s1.union(s2))

jaccard_similarity(set("exponential"), set("exponentia"))
0.8888888888888888
```

- Jaccard distance: $d_{\text{jaccard}}(x, y) = 1 - \frac{|x \cap y|}{|x \cup y|}$

```
def jaccard_distance(s1, s2):
    return 1 - jaccard_similarity(s1, s2)

jaccard_distance(set("exponential"), set("polynomial"))
0.4545454545454546
```

Jaccard Distance

- The Jaccard distance does not fit very well with the sequential nature of strings

```
set1 = set('panmpi')
set2 = set('mapping')
jaccard_distance(set1, set2)
0.16666666666666663
```

```
set1 = set('mapping')
set2 = set('mappin')
jaccard_distance(set1, set2)
0.16666666666666663
```

Jaccard Distance

- Not taking into account order makes the jaccard distance less useful:

```
query = set('guardin')
words = words.words()
distances = compute_distances(query, words)
print("The closest word to query=", query, "is", words[np.argmin(distances)])

closest_words = [words[d] for d in np.argsort(distances)]
print(closest_words[0:10])
```

- Output:

```
The closest word to query= {'u', 'i', 'r', 'n', 'd', 'a', 'g'} is guardian
['unniggard', 'gurniad', 'guarding', 'undaring', 'guardian', 'indiguria', '
    ungrained', 'antidrug', 'unarraigned', 'underguardian']
```

Edit Distance: The Intuition

Problem with Jaccard: it ignores the **order** of characters

Edit distance fixes this: measure how many **operations** are needed to transform one string into the other

Operation	Example
Insert a character	`cat` $\rightarrow$ `ca`$\underline{\texttt{s}}$`t`
Delete a character	`cast` $\rightarrow$ `cat`
Substitute a character	`c`$\underline{\texttt{a}}$`t` $\rightarrow$ `c`$\underline{\texttt{u}}$`t`

Edit distance

The **minimum number of operations** (insert, delete, substitute) needed to transform string x into string y

Edit Distance: Examples

x	y	Operations	Distance
`kitten`	`sitting`	k$\rightarrow$s, e$\rightarrow$i, +g	3
`saturday`	`sunday`	delete a,t; u$\rightarrow$n	3
`cat`	`cut`	a$\rightarrow$u	1
`abc`	`abc`	—	0

Note: unlike Jaccard, `panmpi` and `mapping` are now correctly recognised as more different than `mapping` and `mappin`

Edit Distance: How It Is Computed

The edit distance is computed using **dynamic programming**:

1. Build a matrix C where $C[i, j]$ = edit distance between the first i characters of x and the first j characters of y
2. Fill it left-to-right, top-to-bottom, reusing previously computed values
3. $C[m][n]$ gives the final distance

Complexity: $O(m \cdot n)$ where m, n are the string lengths

Complementary material

The full recurrence, a step-by-step example, Python implementation and efficiency considerations are covered in the companion slides **"Edit Distance"**, available on the course page. You will also work through this in the notebook.

Practical Considerations

Advantages of string distance metrics:

- Fast to compute
- No training required
- Deterministic and interpretable
- Work with any character set/language

Limitations:

- **No semantic understanding**: "car" and "automobile" very distant
- **Language-agnostic**: can be a weakness (no morphology)
- **Sensitive to word order**: "not good" vs "good not"
- Scale poorly for very long strings

String Similarity

From Distance to Semantic Similarity

String Distance:

- Character-level
- Lexical matching
- "car" $\neq$ "automobile"

Semantic Similarity:

- Meaning-level
- Conceptual matching
- "car" $\approx$ "automobile"

From Distance to Semantic Similarity

String Distance:

- Character-level
- Lexical matching
- "car" $\neq$ "automobile"

Semantic Similarity:

- Meaning-level
- Conceptual matching
- "car" $\approx$ "automobile"

Evolution:

- Traditional: TF-IDF + cosine similarity (word overlap)
- Modern: Dense embeddings (capture semantics)

From Distance to Semantic Similarity

String Distance:

- Character-level
- Lexical matching
- "car" $\neq$ "automobile"

Semantic Similarity:

- Meaning-level
- Conceptual matching
- "car" $\approx$ "automobile"

Evolution:

- Traditional: TF-IDF + cosine similarity (word overlap)
- Modern: Dense embeddings (capture semantics)

The goal is to represent sentences in a way that similar meanings have similar representations, regardless of word choice!

The Embedding Revolution

Key idea: Represent text as dense, low-dimensional vectors that capture meaning

Evolution of embeddings:

1. **Word2Vec, GloVe** (2013-2014): Word-level embeddings
   - Problem: How to get sentence embeddings? (averaging loses information)
2. **BERT** (2018): Contextualized word embeddings
   - Problem: Not optimized for similarity tasks
3. **Sentence-BERT (SBERT)** (2019): Direct sentence embeddings
   - Specifically trained for similarity comparisons

Can We Use BERT?

BERT (Bidirectional Encoder Representations from Transformers):

- Based on **Transformer architecture**
- Pre-trained on massive text corpora
- Generates **contextualized embeddings**: same word can have different embeddings

Example:

- "I went to the **bank** to deposit money"
- "I sat by the river **bank**"

The word "bank" gets different embeddings in each context!

Can We Use BERT?

BERT (Bidirectional Encoder Representations from Transformers):

- Based on **Transformer architecture**
- Pre-trained on massive text corpora
- Generates **contextualized embeddings**: same word can have different embeddings

Example:

- "I went to the **bank** to deposit money"
- "I sat by the river **bank**"

The word "bank" gets different embeddings in each context!

Main Problem: BERT was not designed for sentence similarity

Why BERT Alone Isn't Enough

Naive approaches do not work:

1. **Using [CLS] token:** Not trained to capture sentence meaning
2. **Averaging word embeddings:** Loses important information
3. **Computing similarity with BERT requires:**
   - Feeding both sentences through the network together
   - $O(n^2)$ comparisons for n sentences
   - Too slow for real applications!

Why BERT Alone Isn't Enough

Naive approaches do not work:

1. **Using [CLS] token:** Not trained to capture sentence meaning
2. **Averaging word embeddings:** Loses important information
3. **Computing similarity with BERT requires:**
   - Feeding both sentences through the network together
   - $O(n^2)$ comparisons for n sentences
   - Too slow for real applications!

We need a way to get fixed-size sentence embeddings that:

- Capture semantic meaning
- Can be compared efficiently with cosine similarity

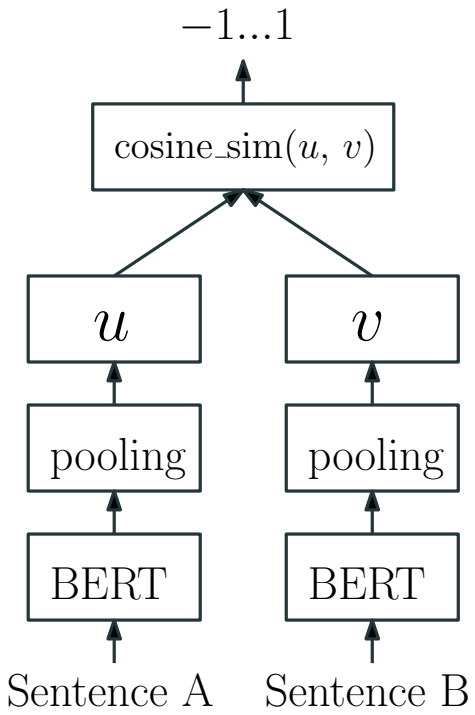
Sentence-BERT (SBERT)

SBERT is generated by fine-tuning BERT using **siamese networks**

Main advantages:

- Generates fixed-size embeddings (e.g., 768-dimensional)
- Embeddings can be compared with simple cosine similarity
- Efficient: encode once, compare many times
- State-of-the-art performance on semantic similarity tasks

SBERT Architecture



Key components:

1. **Shared BERT encoder**: Processes input sentences
2. **Pooling layer**: Aggregates token embeddings (mean, max, or [CLS])
3. **Training objective**: Siamese or triplet network

SBERT Training: Siamese Network

Training with sentence pairs:

1. Input: Sentence pairs with similarity labels
2. Each sentence $\rightarrow$ BERT encoder $\rightarrow$ pooling $\rightarrow$ embedding u, v
3. Compute: similarity(u, v) using cosine
4. Loss: Compare predicted similarity with true label

Training data: Natural Language Inference (NLI) datasets

- Entailment $\rightarrow$ high similarity
- Contradiction $\rightarrow$ low similarity

SBERT Training: Triplet Network

Triplet Loss

Given: anchor sentence a, positive p (similar), negative n (dissimilar)

Goal: distance$(a, p) <$ distance(a, n)

Loss: $\max(0, ||a - p||^2 - ||a - n||^2 + \epsilon)$

Example triplet:

- Anchor: "A dog is running in the park"
- Positive: "A canine is playing outside"
- Negative: "A cat is sleeping on the couch"

This forces the model to learn that semantically similar sentences should be close in embedding space

Using SBERT: Python Implementation

```python
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

# Load pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Encode sentences
sentences = [
    "A dog is playing in the park",
    "A canine is running outside",
    "The cat sleeps on the sofa"
]

embeddings = model.encode(sentences)

# Compute similarity
sim_0_1 = cosine_similarity([embeddings[0]], [embeddings[1]])[0][0]
sim_0_2 = cosine_similarity([embeddings[0]], [embeddings[2]])[0][0]

print(f"Similarity (0,1): {sim_0_1:.3f}")  # around 0.75
print(f"Similarity (0,2): {sim_0_2:.3f}")  # around 0.25
```

Computing Similarity with Embeddings

Cosine Similarity (most common)

$$\text{cosine}(u, v) = \frac{u \cdot v}{||u|| \cdot ||v||} \in [-1, 1]$$

- 1 = identical direction (very similar)
- 0 = orthogonal (unrelated)
- -1 = opposite direction (very dissimilar)

Euclidean Distance

$$\text{distance}(u, v) = ||u - v|| = \sqrt{\sum_{i=1}^{n} (u_i - v_i)^2}$$

- Smaller = more similar

Applications of Semantic Similarity

1. **Semantic Search**
   - Find documents by meaning, not just keywords
   - "ML algorithms" matches "machine learning techniques"
2. **Duplicate Detection**
   - Find similar questions, reviews, support tickets
   - Goes beyond exact or near-exact matches
3. **Clustering & Classification**
   - Group similar documents
   - Few-shot classification using similarity
4. **Recommendation Systems**
   - "Users who liked X also liked Y"
   - Based on content similarity

Comparison: String Distance vs Semantic Similarity

Aspect	**String Distance**	**SBERT**
Level	Character	Semantic
"car" vs "automobile"	Very different	Very similar
Computational cost	Very low	Medium-High
Training required	No	Yes (pre-trained)
Synonyms	Not handled	Handled
Paraphrases	Not handled	Handled
Typo tolerance	High	Medium
Domain adaptation	N/A	Possible
Multilingual	Language-agnostic (any characters)	Needs multilingual models

Evaluation Metrics

Automatically Evaluating NLP Systems

Why automatic metrics matter:

- Human evaluation is expensive and slow
- Need fast feedback during development
- Enable comparison across systems
- Track progress during training

Main difficulties:

- Multiple correct outputs possible
- Different ways to express the same meaning
- Quality is multi-dimensional (fluency, accuracy, relevance)

Types of NLP Tasks Requiring Evaluation

Generation tasks:

- **Machine Translation**: Source language → Target language
- **Summarization**: Long document → Short summary
- **Text Generation**: Prompt → Generated text
- **Dialogue Systems**: User input → Response

Structured prediction tasks:

- **Named Entity Recognition**: Text → Tagged entities
- **Parsing**: Sentence → Syntactic tree
- **Question Answering**: Question + Context → Answer

Reference-based vs Reference-free Metrics

Reference-based metrics:

- Compare system output against **gold-standard references**
- Examples: BLEU, ROUGE, METEOR
- Require human-created reference outputs
- Most common approach

Reference-free metrics:

- Assess quality without reference texts
- Examples: Perplexity, BERTScore (can be reference-free)
- Useful when references are unavailable
- Often measure fluency or coherence

BLEU: Bilingual Evaluation Understudy

BLEU is a widely used metric for **machine translation** evaluation

Core idea:

- Measure n-gram overlap between candidate and reference translation(s)
- Higher overlap = better translation
- Incorporates brevity penalty to avoid short translations

Key properties:

- Score range: $[0, 1]$ (often reported as 0-100)
- Corpus-level metric (computed over entire test set)
- Fast to compute
- Language-independent

BLEU: N-gram Precision

Modified n-gram precision:

For each n-gram in the candidate, count how many times it appears in any reference translation

Example:

- Candidate: "the the the cat"
- Reference: "the cat is on the mat"

Unigram precision (naive): $\frac{4}{4} = 1.0$ Too generous!

Modified unigram precision: $\frac{2}{4} = 0.5$ Clips count to max in reference

$$p_n = \frac{\text{\# of n-grams in the candidate that appear in the reference (clipped)}}{\text{\# of n-grams in the candidate}}$$

BLEU: Complete Formula

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

Where:

- p_n = modified n-gram precision for n-grams of length n
- w_n = weight for each n-gram (typically $w_n = \frac{1}{N}$, uniform)
- N = maximum n-gram length (typically 4)
- BP = brevity penalty

Brevity Penalty (BP):

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases}$$

where c = candidate length, r = reference length

BLEU: Why the Brevity Penalty?

Problem without BP:

- Candidate: "the cat"
- Reference: "the cat is on the mat"

Precision would be perfect (1.0) because all words match!

Solution: Brevity Penalty

- Penalizes translations shorter than reference
- $c = 2$ (candidate length), $r = 6$ (reference length)
- $BP = e^{(1-6/2)} = e^{-2} \approx 0.135$
- Final BLEU will be much lower

This prevents systems from gaming the metric by producing short outputs

BLEU: Calculation Example

Simple example:

- Candidate: "the cat is on mat"
- Reference: "the cat is on the mat"

N-gram precisions:

- Unigrams (1-gram): $p_1 = \frac{5}{5} = 1.0$ (all words appear)
- Bigrams (2-gram): $p_2 = \frac{3}{4} = 0.75$ ("the cat", "cat is", "is on")
- Trigrams (3-gram): $p_3 = \frac{2}{3} = 0.33$ ("the cat is", "cat is on")
- 4-grams: $p_4 = \frac{1}{2} = 0.5$ ("the cat is on")

$BP = e^{(1-6/5)} = e^{-0.2} \approx 0.82$

$\text{BLEU} = 0.82 \cdot \exp(\frac{1}{4}(\log 1.0 + \log 0.75 + \log 0.33 + \log 0.5))$

Result: BLEU ≈ 0.48 (or 48 out of 100)

BLEU: Strengths and Weaknesses

Strengths:

- Fast and easy to compute
- Language-independent
- Correlates reasonably well with human judgment (at corpus level)
- Widely adopted (enables comparison across papers)
- Multiple references can be used

Weaknesses:

- Poor at sentence level
- Ignores semantics
- Doesn't consider word order beyond n-grams
- Penalizes valid paraphrases
- Multiple references help, but rarely available

ROUGE: Recall-Oriented Understudy for Gisting Evaluation

ROUGE is the standard metric for evaluating **text summarization** systems

Key differences from BLEU:

- BLEU focuses on **precision** (what system generated is correct)
- ROUGE focuses on **recall** (coverage of reference content)
- Makes sense: summaries should cover important information!

Multiple ROUGE variants:

- ROUGE-N: N-gram overlap
- ROUGE-L: Longest common subsequence
- ROUGE-W: Weighted longest common subsequence
- ROUGE-S: Skip-bigram overlap

ROUGE-N: N-gram Overlap

Measures n-gram recall:

$$\text{ROUGE-N} = \frac{\#\text{ of n-grams shared by candidate and reference}}{\#\text{ of n-grams in the reference}}$$

Common variants:

- **ROUGE-1**: Unigram overlap (measures content coverage)
- **ROUGE-2**: Bigram overlap (measures fluency)

ROUGE-N: Example

Candidate: “the cat was under the bed”

Reference: “the cat was found under the bed”

ROUGE-1 (Unigrams)

- Matching unigrams: “the” (2×), “cat”, “was”, “under”, “bed” (6x)
- Total unigrams in reference: 7
- Recall: $\frac{6}{7} \approx 0.857$

ROUGE-2 (Bigrams)

- Matching: “the cat”, “cat was”, “was under”, “under the”, “the bed” (5x)
- Total in reference: 6
- Recall: $\frac{5}{6} \approx 0.833$

ROUGE-L: Longest Common Subsequence

ROUGE-L Measure the longest common subsequence (LCS) between candidate and reference

LCS will be a subsequence that appears in both sequences in the same order (but not necessarily contiguous)

Advantages over ROUGE-N:

- Captures sentence-level structure
- No need to predefine n-gram length

Formula:

$$R_{\text{lcs}} = \frac{\text{LCS}(X, Y)}{m}, \quad P_{\text{lcs}} = \frac{\text{LCS}(X, Y)}{n}$$

$$F_{\text{lcs}} = \frac{(1 + \beta^2) R_{\text{lcs}} P_{\text{lcs}}}{R_{\text{lcs}} + \beta^2 P_{\text{lcs}}}$$

where m = reference length, n = candidate length, β typically = 1

ROUGE: Standard Practice in Summarization

Report multiple ROUGE scores:

- **ROUGE-1**: Content coverage (unigram overlap)
- **ROUGE-2**: Fluency (bigram overlap)
- **ROUGE-L**: Overall quality considering structure

Each can report:

- Precision (how much of candidate appears in reference)
- Recall (how much of reference appears in candidate)
- F1-score (harmonic mean)

Most papers report recall and F1 scores for ROUGE-1, ROUGE-2, and ROUGE-L.

ROUGE: Strengths and Weaknesses

Strengths:

- Good for summarization (recall-oriented)
- Multiple variants capture different aspects
- Well-established and widely used
- Correlates with human judgment

Weaknesses:

- **Surface-level matching only**: Misses paraphrases
- **Requires reference summaries**: Expensive to create
- **Can be gamed**: High ROUGE doesn't guarantee quality
- **Doesn't assess factual correctness**
- **Single reference often insufficient**: Multiple references help but rare

BERTScore: Embedding-based Evaluation

Use contextualized embeddings (BERT) to measure semantic similarity

Steps:

1. Encode each token in candidate and reference using BERT
2. Compute cosine similarity between token embeddings
3. Aggregate similarities for precision, recall, F1

Advantages:

- Handles **paraphrases and synonyms** naturally
- Better correlation with human judgment
- Works across different tasks

Disadvantages:

- Computationally expensive
- Requires GPU for reasonable speed

BERTScore: Token-Level Similarity

For each token in candidate $\hat{x} = \{\hat{x}_i\}$ and reference $x = \{x_j\}$:

Recall:

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{x_j \in x} \max_{\hat{x}_i \in \hat{x}} \mathbf{x}_j^T \hat{\mathbf{x}}_i$$

Precision:

$$P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{\hat{x}_i \in \hat{x}} \max_{x_j \in x} \mathbf{x}_j^T \hat{\mathbf{x}}_i$$

F1:

$$F_{\text{BERT}} = 2\frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}$$

where $\mathbf{x}_j$ and $\hat{\mathbf{x}}_i$ are BERT embeddings

BERTScore: Token-Level Similarity

For each token in candidate $\hat{x} = \{\hat{x}_i\}$ and reference $x = \{x_j\}$:

Recall:

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{x_j \in x} \max_{\hat{x}_i \in \hat{x}} \mathbf{x}_j^T \hat{\mathbf{x}}_i$$

Precision:

$$P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{\hat{x}_i \in \hat{x}} \max_{x_j \in x} \mathbf{x}_j^T \hat{\mathbf{x}}_i$$

F1:

$$F_{\text{BERT}} = 2\frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}$$

where $\mathbf{x}_j$ and $\hat{\mathbf{x}}_i$ are BERT embeddings

NOTE: for Recall and Precision, a greedy approach is followed, where each token is matched to the most similar token in the other sentence

Perplexity: Language Model Quality

Measures how well a language model predicts a sample of text

Formula:

$$\text{Perplexity}(W) = P(w_1, w_2, ..., w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1, w_2, ..., w_N)}}$$

or equivalently:

$$\text{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i|w_1, ..., w_{i-1})\right)$$

Interpretation:

- Lower perplexity = better model
- Roughly: average number of equally likely choices at each step
- PPL = 50 model as uncertain as if choosing uniformly from 50 words

Perplexity: Usage and Limitations

When to use:

- Evaluating language models during training
- Comparing different LM architectures
- Intrinsic evaluation (not task-specific)

Limitations:

- **Not directly interpretable**: What's a *good* perplexity?
- **Doesn't measure generation quality**: Low perplexity doesn't guarantee good outputs
- **Only applicable to models with probability outputs**
- **Domain-dependent**: Can't compare across different corpora

Metrics for Structured Outputs

1. **Named Entity Recognition (NER)**
   - Precision, Recall, F1 at entity level
2. **Part-of-Speech Tagging**
   - Token-level accuracy
3. **Parsing**
   - Compare predicted and reference parse trees
   - F1 over tree constituents (bracketing F1)
4. **Question Answering**
   - Exact Match (EM): exact string match
   - F1 over tokens: Partial credit

Choosing the Right Metric

Task	**Primary Metrics**
Machine Translation	BLEU, METEOR, (BERTScore)
Summarization	ROUGE-1, ROUGE-2, ROUGE-L
Text Generation	Perplexity, Human eval, (BERTScore)
NER	Entity-level F1
POS Tagging	Token accuracy
Parsing	Parsing & Bracketing F1
Extractive QA	EM, Token F1
Classification	Accuracy, Precision, Recall, F1

Limitations of Automatic Metrics

1. **Multiple valid outputs**
   - Many ways to translate, summarize, or answer
   - Metrics penalize valid alternatives
2. **Surface-level matching**
   - Most metrics ignore semantics
   - Even BERTScore has limitations
3. **Don't measure all quality dimensions**
   - Factual correctness
   - Coherence and discourse structure
   - Appropriateness for context
4. **Can be gamed**
   - Systems optimized for metrics, not actual quality

Human Evaluation: The Gold Standard

Why human evaluation matters:

- Only humans can truly judge semantic quality
- Can assess multiple dimensions simultaneously
- Catch subtle errors automatic metrics miss

Common human evaluation methods:

1. **Absolute scoring**: Rate on Likert scale (1-5)
   - Adequacy: Does it convey the meaning?
   - Fluency: Is it grammatical and natural?
2. **Comparative ranking**: Which output is better? (A vs B)
3. **Error analysis**: Categorize and count error types
4. **Task-based evaluation**: Can humans accomplish a task using the output?

Summary

Summary: Three Approaches to Text Comparison

	Level	**Methods**	**Speed**	**Use cases**	**Semantics**
String Distance	Character	Edit distance	Very fast	Typos, OCR, fuzzy matching	No
Semantic Similarity	Meaning	SBERT, embeddings, cosine similarity	Medium	Search, clustering, paraphrases	Yes
Evaluation Metrics	Task-specific	BLEU, ROUGE, METEOR, BERTScore	Fast to Medium	Translation, summarization, QA	Varies

Decision Guide: When to Use What

Use String Distance when:

- Detecting typos, spelling errors, OCR mistakes
- Fuzzy matching in databases (names, addresses)
- Speed is critical and semantics don't matter

Use Semantic Similarity when:

- Meaning matters more than exact wording
- Semantic search, document retrieval
- Clustering by topic, paraphrase detection

Use Evaluation Metrics when:

- Assessing quality of NLP system outputs
- Benchmarking models (translation, summarization, etc.)